

Football Analytics API – Technical Report

Module: COMP3011

Project: REST API + minimal frontend for football analytics and favourite player lists.

Links (for submission):

- **GitHub repository:** *[Add your public repo URL here, e.g. <https://github.com/username/football-analytics-api>]*
 - **API documentation:** [API documentation \(PDF\)](#) in repo; interactive docs at `/docs` (Swagger UI) and `/redoc` when the app is running (or at `live-url/docs` if deployed).
 - **Presentation slides:** *[Add link to your slides, e.g. Google Drive or OneDrive]*
-

1. Technology stack and justification

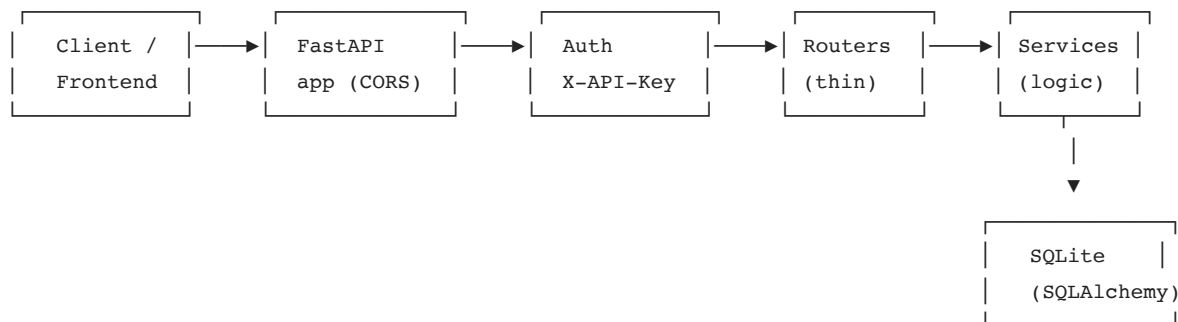
Layer	Choice	Justification
Backend	Python 3.10+, FastAPI	Async support, automatic OpenAPI/Swagger, Pydantic validation, clear routing and dependency injection.
ORM	SQLAlchemy 2.x	Mature ORM with declarative models, session management, and FK support; fits the ERD and CSV-derived schema.
Validation	Pydantic	Request/response schemas, clear validation errors (422), type-safe config via pydantic-settings.
Database	SQLite	Single-file, no separate server; suitable for coursework and local deployment; can be swapped for PostgreSQL via <code>DATABASE_URL</code> .
Testing	pytest, FastAPI TestClient	In-process API tests, fixtures for client and auth headers; no need for httpx async for basic status-code and JSON tests.
Frontend	Vanilla HTML, CSS, JS	No framework; fetch API with X-API-Key; minimal scope as per brief (players list, favourite lists)

Layer	Choice	Justification
		CRUD, add/remove players).

Environment configuration is via `.env` and `pydantic-settings` ; no secrets in code. The API key is validated in a single dependency and applied to all `/api/v1/*` routes.

2. Architecture

Request flow is layered: **Client** → **FastAPI app** → **Auth (X-API-Key)** → **Routers** → **Services** → **Database**. Routers are thin (parse request, call service, map exceptions to HTTP); business logic lives in services and receives the DB session via dependency injection.



Key components:

- **app/main.py**: App factory, CORS, mount of API v1 router and frontend at `/app` ; `/health` and `/docs` remain on the main app.
- **app/auth**: Single dependency that reads `X-API-Key` , compares to config, returns 401 with a consistent JSON body if missing or invalid.
- **app/routers**: One router per resource (players, teams, favourite_lists, analytics); all under `/api/v1` with the auth dependency.
- **app/services**: Reusable functions (e.g. `get_players` , `get_top_scorers`) that take a DB session; no HTTP concerns.
- **app/models / app/schemas**: SQLAlchemy models for dataset and application tables; Pydantic schemas for request/response and pagination.

The dataset (players, teams, performances, transfers, market values) is read-only from the API; the only full CRUD resource is **FavouriteList**, with **FavouriteListPlayer** used for many-to-many membership (add/remove/view list players).

3. Design choices

REST and URLs: All API endpoints are under `/api/v1`. Resource-based paths: e.g. `GET /api/v1/players`, `GET /api/v1/players/{id}/performances`, `POST /api/v1/favourite-lists`, `GET /api/v1/favourite-lists/{id}/players`.

Status codes: 200 (OK), 201 (Created for POST), 204 (No Content for DELETE), 400/422 (validation), 401 (missing/invalid API key), 404 (resource not found), 409 (e.g. player already in list). Error responses use a consistent JSON shape (e.g. `{"detail": {"message": "...", "code": "..."} }`).

CRUD resource: The main database-backed CRUD resource is **FavouriteList** (create, read list, read one, update name, delete). `FavouriteListPlayer` supports only add player, remove player, and list players (with optional sort); it is not a separate full CRUD resource.

Pagination: List endpoints that return many items (e.g. players, performances, transfers) use `page`, `limit`, and respond with `{ "data": [...], "page", "total_pages", "total_count" }` .

Authentication: Every `/api/v1/*` request must include a valid `X-API-Key` header; the dependency runs before route handlers. `/health`, `/docs`, and `/openapi.json` are unauthenticated.

4. Implementation highlights

Sorting (players and list players): The API supports `sort_by` (e.g. `name`, `age`, `market_value`, `minutes_played`) and `order` (`asc/desc`). Age is derived from `date_of_birth`; market value from the latest row per player in `player_market_value` (subquery on max date); minutes from `SUM(minutes_played) OVER player_performances`. The same logic is used for listing players inside a favourite list.

Analytics: Three endpoints demonstrate use of the football dataset: (1) **top-scorers** – `SUM(goals)` from performances, optional filter by season/competition_id; (2) **top-market-values** – latest value per player, ordered by value desc; (3) **most-minutes-played** – `SUM(minutes_played)` per player, optional season filter. All return a list of player identifiers and the computed metric.

Advanced analytics:

- **Player details** – `GET /api/v1/players/{id}/details` returns current market value, full market value history, and a career summary. Current market value is the latest value per player from `player_market_value` (row with max date per player). Market value history is all rows for that player ordered by date. Career summary includes seasons played (distinct seasons from `player_performances`) and previous clubs (distinct team names from performances joined with teams, or from transfer history). One combined response keeps the frontend modal simple.
- **New analytics endpoints – top-assists:** sum of `assists` per player from `player_performances`, with optional season and competition filters, returned in descending order. **Youngest-stars:** players under a configurable age limit, with aggregated `total_minutes` and `total_goals` from performances, sorted by minutes then goals. Both use the same `PlayerPerformance` (and player) data as the existing analytics endpoints.

Alternatives considered:

- **REST vs GraphQL:** REST was chosen for simplicity and strong tooling (OpenAPI, Swagger/ReDoc). GraphQL would suit flexible client queries but was not required for this scope.
- **SQLite vs PostgreSQL:** SQLite for single-file, no separate server, and easy local use; for production a server DB such as PostgreSQL with migrations (e.g. Alembic) would be preferred.
- **One details endpoint vs several:** A single combined details endpoint was chosen for the player modal. Splitting into e.g. `/market-value-history` and `/career` would allow finer-grained caching but was not needed for the current frontend.

Dataset pipeline: Data is filtered to top-five leagues (GB1, ES1, IT1, L1, FR1) in a dedicated script that builds allowed club/player IDs and writes filtered CSVs to `web/filtered/`. A separate import script loads these in FK order (teams → players → performances, transfers, market values) with batched commits and streaming reads to avoid loading entire files into memory.

5. Testing approach

Tests use `pytest` and FastAPI's `TestClient` (no separate server). **conftest.py** provides a shared client and `auth_headers` (valid API key). The same SQLite database used for local development is used for tests (no in-memory override in the current setup).

Coverage: (1) **Auth** – request without key → 401; wrong key → 401; valid key → 200 on a protected route. (2) **Favourite lists CRUD** – full cycle (create, get, patch, delete, then 404); 404 for get/patch/delete on missing id. (3) **Favourite list players** – add player, list players, add same again → 409, remove player, list empty; 404 when list or player does not exist. (4) **Validation** – POST favourite list with empty or missing name → 422; add player with non-existent player_id → 404. (5) **Players** – list returns 200 and structure `data , page , total_pages , total_count ; sort_by=name; get by id 200/404`. (6) **Analytics** – GET top-scorers (and other analytics endpoints) returns 200 and list of items with expected fields.

Tests are run with `pytest` or `pytest tests/ -v`; see README.

6. Deployment

Deployment has **not** been performed for this submission. The project runs locally with SQLite. The README describes how to run the server (`uvicorn app.main:app`), create the DB, load data, and run tests. When deployment is performed, replace this paragraph with one or two sentences: where the app is deployed (e.g. PythonAnywhere or Render), that `DATABASE_URL` and `API_KEY` are set, and that `create_db` and `load_data` were run once. Add the live API URL (and frontend URL if applicable).

7. Use of Generative AI

Generative AI (e.g. Cursor/Claude) was used for design discussion, code generation, and report structure. All suggestions were verified and adapted; code and text were reviewed and edited as needed. Details and example excerpts are documented in the GenAI declaration (see appendix or `docs/GENAI_DECLARATION.md`). This section links the technical report to that declaration.

8. Limitations and future work

- **Single API key:** Authentication is a single shared key; there is no per-user identity or rate limiting.
- **SQLite:** Suitable for coursework and small scale; for production, a server DB (e.g. PostgreSQL) and proper migrations (e.g. Alembic) would be preferable.
- **Frontend:** Minimal by design; improvements could include better error feedback, loading states, and responsive layout.

- **Data:** Dataset is static after import; no live sync with an external source. Analytics are computed at request time (subqueries/aggregations); materialized views or cached aggregates could improve performance for heavy use.
-

Appendix A: GenAI Declaration

The following is the Generative AI tools declaration, included in this report for the single-PDF submission.

1. Tools used and purpose

Tool / platform	Purpose
Cursor (AI-assisted editor)	Design and architecture discussion; code generation for FastAPI routers, Pydantic schemas, SQLAlchemy models, and service layer functions; refactoring (e.g. moving from mounted sub-app to single app with router dependencies).
Claude / Codex (via Cursor)	Writing and editing Python (app code, scripts, tests); drafting README and API descriptions; suggesting test cases and validation behaviour (e.g. 422 for empty name, 404 for invalid player_id).
Other	None.

2. Sample conversation logs (appendix)

Excerpt 1 – API key dependency and router setup

I asked how to require X-API-Key on all `/api/v1` routes. The suggestion was to add a dependency (e.g. `require_api_key`) that reads the header and returns 401 if missing or invalid, and to attach it to the API v1 router via `APIRouter(..., dependencies=[Depends(require_api_key)])`. I applied this so every route under `/api/v1`` is protected without repeating the check in each handler.

Excerpt 2 – Player details endpoint design

I asked how to return current market value, full market value history, and a career summary from one endpoint. The suggestion was a single `GET /players/{id}/details` response with: current value as the latest row per player in `player_market_value` (max date); history as all rows for that player ordered by date; career with seasons played

(distinct from performances) and previous clubs (distinct team names from performances/teams or transfers). I implemented the service and Pydantic response schema (e.g. `PlayerDetailsResponse` with `market_value_history`, `career`) as suggested and wired the router.

Excerpt 3 – Top assists and youngest stars queries

I requested analytics endpoints for top assists (sum of assists per player, optional season/competition filters) and youngest stars (players under an age limit, aggregated minutes and goals, sorted by minutes then goals). I was given SQLAlchemy query patterns (group by player, join performances/players, filter by age from `date_of_birth`). I implemented the routes and response schemas (e.g. `TopAssistsResponse`, `YoungestStarResponse`) and added the same optional query parameters as the existing analytics endpoints.

Excerpt 4 – Test cases for new endpoints

I asked for tests for the new player details and analytics endpoints without changing the test DB. The suggestion was: (1) `test_player_details_ok` – GET `players?limit=1`, take first id, GET details, assert 200 and presence of `player_id`, `player_name`, `current_market_value`, `market_value_history`, `career` (`seasons_played`, `previous_clubs`); (2) `test_player_details_404` for id 999999999; (3) `test_top_assists` and `test_youngest_stars` – GET with limit, assert 200 and list shape, and if non-empty assert first item keys and optionally descending order. I added these to `test_players.py` and `test_analytics.py` and ran `pytest` until green.

3. Reflection on "creative, high-level" use

I used GenAI for both high-level design (e.g. REST structure, endpoint design, analytics ideas) and implementation (code, tests, documentation). I checked and adapted all suggestions—for example around security (API key handling), error handling (401/404/422), and validation (request/response schemas)—and ran tests to confirm behaviour. I understand and can explain all code and design choices in this submission.

Report length: ~5 pages when rendered to PDF (main report) plus appendix. Export this Markdown to a single PDF (e.g. via Pandoc or print-to-PDF) for Minerva submission.